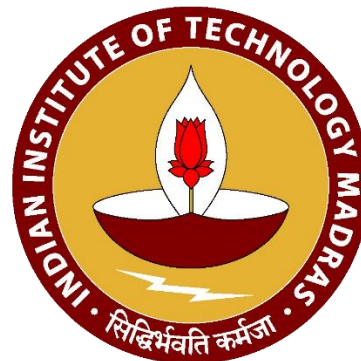


Python for Data Science

Web M. Tech Course DA5301W

Modules 6&7: Pandas – Part 2

Dr. P.S Jayadev



Contents

- Datetime datatype in Python
- Handling Datetime in DataFrames
- Sorting and Ranking Dataframes
- Descriptive Statistics on Dataframes
- Combining and Reshaping Dataframes
- Dropping rows and columns

Date and Time Data in Python

- Many real-world datasets involve date and time:
 - Sales or production logs, machine events
 - Sensor and IoT readings
 - Website visits
 - Financial transactions, e-commerce transactions
- Date and Time-based data helps analyze:
 - Trends and seasonality
 - Durations and delays
 - Performance over time
- Python has an in-built 'datetime' module for representing and manipulating date and time

Datetime module in Python

Class	Purpose	Example
date	Calendar date (year, month, day)	<code>datetime.date(2024, 5, 17)</code>
time	Time of day (hour, min, sec)	<code>datetime.time(14, 30, 0)</code>
datetime	Combination of both	<code>datetime.datetime(2024, 5, 17, 14, 30, 0)</code>
timedelta	Difference between two datetimes in weeks, days, hours, minutes, seconds, milliseconds, microseconds	<code>datetime.timedelta(days=7)</code>

- Format codes like %Y, %m, %d, %H, %M, %S represent year, month, etc. for converting from string to datetime objects and vice-versa
- Arithmetic operations can be performed by adding timedelta to datetime objects
- Datetime objects are comparable and sortable

Handling Datetime in DataFrames

- When reading from CSV or Excel, date columns are often read as string objects
- Two options:
 - Parse date columns directly while reading from file using `parse_date` argument
 - **Note:** Columns already stored as dates in xlsx format are auto-parsed
 - Convert them to pandas datetime object after reading using `pd.to_datetime`
- `.dt` is to be used to access and manipulate various datetime properties and methods in pandas

Handling Datetime in DataFrames

- Types of datetime in pandas:
 - `datetime64[ns]`
 - Default datetime type in pandas
 - Stores absolute timestamps without timezone information
 - ns implies nanosecond precision
 - Fast and efficient for time-series data within a single timezone
 - `datetime64[ns, tz]`
 - Stores timestamps with an associated timezone
 - Internally stored in UTC, but displays in local timezone
 - Useful for global datasets with timezones (e.g., financial logs, globally spread IoT devices, etc.)

Generating DateRanges in Pandas

- `pd.date_range()` is used to create a sequence (range) of evenly spaced datetime values
- Commonly used to:
 - Create **time-based indexes**
 - Generate **date sequences** for simulation or filling missing dates known to be in an order
- `pd.date_range()` can be passed to `df.set_index()` to set datetime index to a dataframe

Sorting and Ranking Dataframes

Sorting and Ranking Dataframes

- **Sorting:** Rearranges data in ascending or descending order based on specific values or labels
- Returns a new dataframe with values sorted by the chosen column
- Sorting can be based on multiple columns given as a list
- Two methods of sorting exist in pandas:
 - `sort_values()` : Sorts data by row or column values
 - `sort_index()` : Sort data by index values

Sorting and Ranking Dataframes

- **Ranking:** Assigns numerical ranks to values in a column or row based on their relative values (no re-arrangement happens)
- A new Series (or a column added to a DataFrame) containing the rank position (1st, 2nd, 3rd, etc.) of each value
- Ranks can be fractional values as well when handling ties and it can be given a method as to how ties are to be handled
 - 'average' (default): Assigns the mean of the ranks that would have been assigned to all the tied values
 - 'min': Assigns the smallest rank among all tied values
 - 'max': Assigns the largest rank among all tied values
 - 'dense': Ranks are consecutive numbers, with ties getting the same rank, but the next value gets the next consecutive rank (no gaps)

Sorting and Ranking Dataframes

- Need for Sorting and Ranking in Data Science:
 - To organize large datasets to quickly identify trends
 - Essential for exploratory data analysis (EDA) — finding top customers, highest sales, etc.
 - Supports feature engineering, such as rank-based transformations

Descriptive Statistics on Dataframes

Descriptive Statistics on DataFrames

- Pandas provides quick, built-in functions for:
 - Central tendency (mean, median, mode)
 - Spread (standard deviation, variance, IQR)
 - Relationships (correlation)
- `df.describe()` gives summary statistics for all the numerical columns in the dataframe
- Mention argument (`include='all'`) for including categorical columns as well
- Other methods: `df['col'].mean()`, `df['col'].std()`, `df.corr()`

Combining Dataframes – Merge, Join, Concat

Combining Dataframes

- Motivation:

- Real-world data often comes from multiple files or tables
- Combining lets us build a unified dataset for analysis

- Concatenate DataFrames – `pd.concat()`:

- Generally used when DataFrames have the same columns
- Vertically stacks dataframes by default
- Horizontal stacking is also possible by setting `axis=1`
- If reindexing is to be done after concatenation, pass `ignore_index=True`

Combining Dataframes

- Merge DataFrames:

- Used when DataFrames share common columns or keys
- Equivalent to SQL join
- Column(s) or index on which merging is to be done is to be specified – merges based on values (like keys in SQL) in those columns
- How merging is to be done is also to be specified
- Eg: `pd.merge(df_orders, df_customers, on="CustomerID", how="left")`
- Missing values show up as NaN

Type	Description
Inner (default)	Matching keys only
left	All from left, match from right
right	All from right, match from left
outer	All keys from both sides

Combining Dataframes

- Join DataFrames:

- Convenient shorthand for merging based on index
- Useful when your DataFrames have meaningful index labels and can be faster for large index-based joins

Reshaping Dataframes – Melt and Pivot Table

Reshaping Dataframes – melt()

- Real-world data may not always be in the form best suited for analysis or visualization
- Reshaping helps you switch between:
 - **Wide format** - multiple columns for values of variables
 - **Long format** - one column for variable names and one for values
- **melt()**: Used to convert wide dataframe to long format
- Takes 3 main arguments:
 - **id_vars**: Columns listed here remain unchanged in the reshaped DataFrame
 - **var_name**: Name of the column that shows names of melted columns (variables)
 - **value_name**: Name of the column that shows the datavalues corresponding to each of the melted columns (variables)
 - **Note**: If you don't specific id_vars, all columns are melted

Reshaping Dataframes – pivot_table()

- **pivot_table()**: Used to convert long dataframe to wide format (like spreadsheet pivot)
- Takes following main arguments:
 - **values**: Specifies which column's data should be aggregated or summarized. If not given, all numeric columns are aggregated by default
 - **index**: Defines which column(s) form the rows (index) in the pivoted table
 - **columns**: Defines which column(s) form the new column headers in the pivot table
 - **aggfunc**: Defines how to combine multiple rows with the same index/column pair and mean is default function
 - **margins**: Setting it to true adds row/column totals to the pivot table

Dropping Rows or Columns

Dropping Rows or Columns

- `df.drop()` can be used to drop both rows and columns specifically
- Dropping requires specification of index or column labels
- **Dropping duplicates:**
 - Remove repeated rows based on all or specific columns
 - You have an option to keep the first or last instance of the duplicate row
 - Use `ignore_index=True` to reset the row indices after dropping